Meter Reading Push to Core Billing

Fix, Refactor & Handover Report

Project: Nobleus Admin / Billing Module
Date: 7 July 2026
Prepared for: Technical Team & Management
Status: **External Blocker**

1. Executive Summary

The billing module was incorrectly reporting that meter readings had been successfully posted to the core billing server even when the remote server returned a database connection error. This caused ledger entries to be marked as **pushed** when they had actually **failed**, resulting in lost readings and unreliable retry logic.

What was fixed:

- The single-reading post and the bulk “Post to Billing” action now share the same reliable client (`CoreBillingClient`) and the same response interpretation logic.
- The client now treats any non-success JSON or non-JSON response as a failure, even when the HTTP status is 200.
- The wrongly-marked ledger entry (Reading #60672) has been corrected from `pushed` to `failed` .
- The missing `billing_audit_logs` table was created from the existing migration schema, so audit logging now works.
- Fallback descriptions for readings not taken on the 28th of the month are preserved.

Remaining blocker: The external core billing server (`reading.hargeisawatertech.com`) is still returning *“Database connection failed”*. Until the remote database is restored, readings will be queued for retry rather than successfully pushed.

2. Problem Statement

2.1 Observed Symptoms

- Users clicked **“Post to Billing”** and saw the message *“Reading posted to billing.”*
- Behind the scenes, the core billing server responded with HTTP 200 but its body contained an error such as `Database connection failed`.
- The application only checked the HTTP status code (2xx = success), so it recorded the push as successful.
- The bulk “Post to Billing” action used an old endpoint (`core.hwa-smartmetering-app.com`) and a different code path than the single-reading post.
- The `billing_audit_logs` table did not exist, so audit records could not be written.

2.2 Business Impact

- Clerks believed readings were delivered when they were not.
- Invoices could not be generated from readings that were never actually posted.
- Retry logic did not trigger because the local state was `pushed` instead of `failed`.
- Operational reporting was inconsistent between single and bulk posting workflows.

3. What Was Fixed

Single reading post	Now uses `Yii::$app->coreBilling`, creates a `BillingLedger` entry, records push response, and marks the raw reading as `STATUS_IMPORTED`.
Bulk “Post to Billing”	Refactored from custom cURL helper to `CoreBillingClient`. Now validates configuration, skips duplicates/zero readings, creates ledger entries, records audit logs, and distinguishes posted vs failed vs skipped counts.
Response interpretation	`CoreBillingClient::postMeterReading` now parses the JSON body. Only responses with `success: true` are treated as success; non-success JSON and non-JSON bodies (including DB errors) are treated as failure even on HTTP 200.
Ledger correction	Reading #60672 (supply `A - 10 - 229`) was corrected from `pushed` to `failed` and its `pushed_at` timestamp was cleared.
Audit logging table	The missing `billing_audit_logs` table was created using the existing migration schema. Audit records are now persisted.
Fallback description	`postMeterReading` accepts an optional `$description` parameter so bulk imports can note when the reading used was not from the 28th of the month.

4. Current State

4.1 Database Snapshot

Verified at the time of this report:

Billing Ledger (latest entries)

3	60672	A - 10 - 229	101,825		*NULL*	2026-07-07

Billing Audit Logs

5	A - 10 - 229	2026-07-07 22:57:32
4	A - 10 - 229	2026-07-07 22:57:09
3	A - 10 - 229	2026-07-07 22:48:44

4.2 Code Changes

The following files were modified or added:

`common/components/CoreBillingClient.php`	Improved response parsing; added optional description parameter.
`backend/modules/billing/controllers/ReadingsController.php`	Refactored bulk post; added single `actionImport` / `actionReject`; removed old `sendToBilling()` helper.
`backend/config/main.php`	Restricted billing domain to billing routes; disabled duplicate jQuery bundle.
`backend/modules/billing/views/readings/view.php`	Updated action buttons/messages.
`backend/modules/billing/views/reports/import-status.php`	Updated status display.
`backend/views/layouts/dashboard/_header.php`	Header/navigation adjustments.
`backend/assets/AppAsset.php`	Asset bundle adjustments.
`deploy/api.hargeisawatertech.com.conf`	Added deployment config.
`deploy/billing.hargeisawatertech.com.conf`	Added deployment config.
`deploy/SETUP.md`	Updated setup documentation.

5. Technical Details

5.1 New Response Logic

`CoreBillingClient::postMeterReading` now returns a structured result:

```
[
 'success'   => bool,   // true only when remote returns success: true
 'reason'    => ?string,// already_posted, same_reading, etc.
 'message'   => string, // human-readable outcome
 'http_code' => int,    // raw HTTP status
 'body'      => mixed,  // parsed JSON or raw text
]
```

5.2 Why HTTP 200 Can Still Be a Failure

The remote server is a PHP application that often emits error text (e.g., *"Database connection failed"*) while the web server still returns HTTP 200. The previous code only checked the HTTP status. The new code inspects the body:

- If the body is valid JSON with `success: true` → success.
- If the body is valid JSON without `success: true` → failure, message taken from `error` / `message`.
- If the body is not valid JSON → failure, message is the raw body text.

5.3 Bulk Post Workflow

1. Load all readings on or before the 28th of the selected month, newest first.
2. For each supply, take the latest reading and convert liters → m³.
3. Skip zero readings and already-ledgered readings.
4. Create a `BillingLedger` entry with previous/current readings and consumption.
5. Call `CoreBillingClient::postMeterReading`.
6. Update the ledger with push status, timestamp, and response body.
7. Write an audit log (`bulk_import` or `bulk_import_push_failed`).
8. Mark the raw reading as `STATUS_IMPORTED` (failed pushes are retried later by the console command).
9. Show a summary message: posted / failed (queued for retry) / skipped.

5.4 Configuration

The active core billing configuration is now driven by these environment variables:

`CORE_READING_API_URL`	`https://reading.hargeisawatertech.com`
`CORE_METERS_API_URL`	`https://meters.hargeisawatertech.com`
`CORE_API_TOKEN`	JWT service token
`BILLING_PORTAL_URL`	`https://billing.hargeisawatertech.com`

The old `BILLING_API_URL` / `BILLING_API_TOKEN` values pointing to `core.hwa-smartmetering-app.com` are no longer used by the reading push code.

6. Remaining External Blocker

> **Remote core billing server is down at the database layer.**
>
> Endpoint: `POST https://reading.hargeisawatertech.com/bill/api/meter-reading`
> Response body: *Database connection failed*
> HTTP status: 200 (body contains error text)

Because the new client correctly classifies this as a failure, every push attempt from the Nobleus side will be recorded as `failed` and queued for the retry console command. The application side is now behaving correctly; no readings will be falsely marked as pushed.

7. Recommendations & Next Steps

Remote API / DevOps	Restore database connectivity on `reading.hargeisawatertech.com` and confirm the `/bill/api/meter-reading` endpoint returns `{"success": true, ...}`.	
Technical Team	Run the billing-push console retry command once the remote DB is restored to flush the queued failed pushes.	**High**
Technical Team	Review and commit the modified files. Run regression tests on single and bulk post flows.	**High**
Management	Communicate to clerks that push failures are now accurately reported and will be retried automatically; avoid manual re-entry until the retry command confirms success.	
Technical Team	Ensure application log files are writable or forwarded to a central location so future issues can be diagnosed quickly.	

8. Appendix: Sample API Response

Example of the remote error that is now correctly treated as a failure:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

Database connection failed
```

9. Appendix: Verification Commands

Latest ledger entries	`SELECT id, raw_reading_id, supply_no, current_reading, push_status FROM billing_ledger ORDER BY id DESC LIMIT 20;`
Recent audit logs	`SELECT id, action, supply_no, created_at FROM billing_audit_logs ORDER BY id DESC LIMIT 20;`
Remote API health	`curl -X POST https://reading.hargeisawatertech.com/bill/api/meter-reading -H "Authorization: Bearer <token>" -d '{"date":"2026-07-07","supplyno":"A - 10 - 229","reading":101}'`